Clase 6 — Taller de retardos (todo resuelto)

Bloque: A (Fundamentos) · **Semana:** 2 · **Duración estimada:** 120 min **Correlativa:** Clase 5 (el retardo en las redes de paquetes) · **Sigue con:** Clase 7 (Nyquist, Shannon y la capa física) **Material de referencia:** Tanenbaum cap. 1 §1.5 · Exámenes `sirc1103` (P5), `sirc0912` (P1), `sirc1002` (P5) · RFC 7323, RFC 9000

> **Esta clase es un taller puro.** No hay teoría nueva: los cuatro componentes ya están en la Clase 5. Acá hay **aritmética con la cuenta a la vista**, y el **Ejercicio 3**, que es el problema de retardos del curso.

🎯 Al terminar esta clase vas a poder

1. **Copiar de memoria** la caja de las cuatro fórmulas, con las unidades de cada variable.
2. **Ejecutar un cálculo de retardo** de punta a punta con `n` enlaces y `k` conmutadores, escribiendo cada término.
3. **Aplicar las tres reglas de sensibilidad**: qué pasa si duplico `L`, `R` o `d`.
4. **Contar bien los** `d_proc`: por qué hay `n` términos de transmisión y solo `n − 1` de procesamiento.
5. **Explicar por qué un conmutador que reenvía bit a bit no aporta retardo de transmisión**, y nombrar ese modelo.
6. **Calcular el *bandwidth-delay product*** y decidir si una ventana de TCP alcanza o no.
7. **Distinguir latencia de RTT** y calcular el **jitter**.
8. **Resolver entero** el Examen `2011-03 Pregunta 5` (10 pts).

> **Prerequisito del examen:** `2011-03 P5` (10 pts — retardo end-to-end con 3 enlaces y 2 conmutadores, y el caso cut-through), `2009-12 P1` (10 pts, Clase 5 §3.1), `2011-03 P2` (6 pts, Clase 5 §3.2), `2010-02 P5a` (8 pts — jitter vs. retardo).

PARTE 1 — LA CAJA DE FÓRMULAS

1.1 Las cuatro fórmulas, para copiar

Esto es lo único que tenés que saber de memoria de la parte de retardo:

```
┌──────────────────────────────────────────────────────────────┐
│   d_transmisión = L / R      L en BITS,   R en bps del enlace     │
│   d_propagación = d / s      d en METROS, s en m/s del medio      │
│   d_cola        = lo que toque    el único componente VARIABLE    │
│   d_proc        ≈ despreciable   CRC + cabecera + elegir puerto  │
│                                                                   │
│   d_end2end = Σ (d_proc + d_cola + L/R + d/s)                     │
│                                                                   │
│   n enlaces  ⟶  n · d_transmisión  +  n · d_propagación           │
│   k routers  ⟶  k · d_proc         +  k · d_cola                  │
│   En una línea sin ramales:  k = n − 1                            │
│   ⚠️ d_proc va (n − 1) VECES, no n veces. Ese es el error del P5'11.│
└──────────────────────────────────────────────────────────────┘
```

1.2 La tabla de unidades: donde se pierden los puntos

El error más caro del curso no es conceptual, es de **unidad**:

Símbolo	Qué es	Unidad	Error típico
`L`	longitud del paquete	**BITS**	poner bytes: factor **8**
`R`	velocidad de transmisión = ancho de banda del enlace	**bit/s**	—
`d`	longitud del enlace físico	**METROS**	poner km: factor **1000**
`s`	velocidad de propagación en el medio	**m/s**	—
`d_cola`	espera en la cola	s / ms	el único no calculable

```
bytes → bits : × 8     km → m : × 1000     µs → ms : ÷ 1000
cobre / fibra óptica : s ≈ 2 · 10⁸ m/s    ← el 95 % de los problemas
aire / espacio libre : s ≈ 3 · 10⁸ m/s    (y un satélite va por aire)
```

1.3 Las tres reglas de sensibilidad del taller

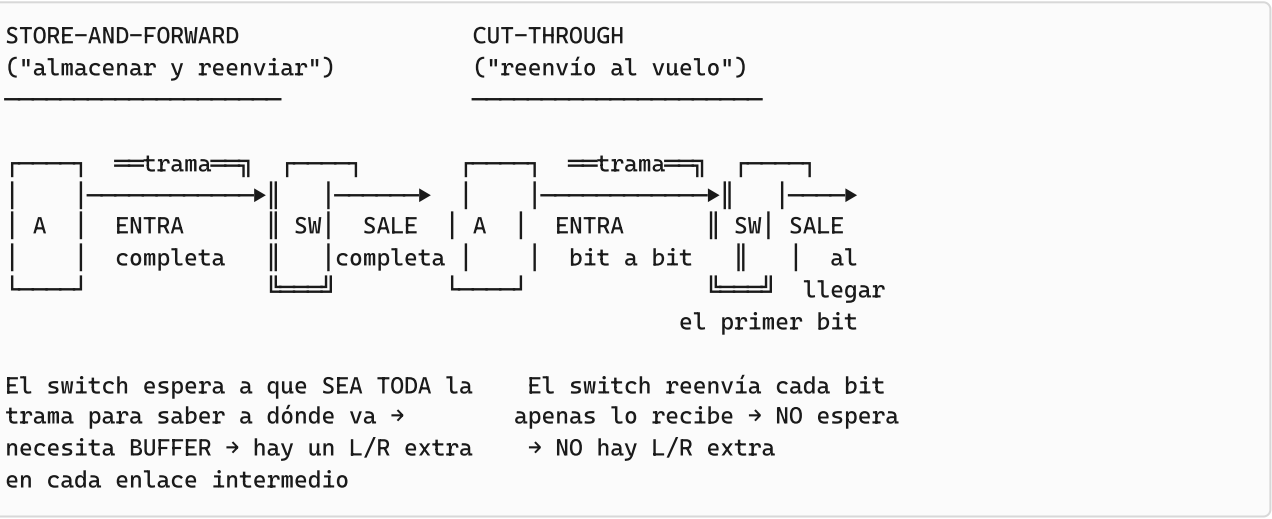
De mirar las fórmulas salen las tres reglas, y con ellas se responde la mitad de los exámenes de esta parte:

Si duplico…	¿Qué cambia?	¿Qué NO cambia?
la longitud `L` del paquete	**solo** `d_transmisión` (se duplica)	`d_propagación`, `d_proc`, `d_cola`
el ancho de banda `R`	**solo** `d_transmisión` (se reduce a la **mitad**)	`d_propagación`
la distancia `d`	**solo** `d_propagación` (se duplica)	`d_transmisión`

> **Las dos frases que hay que dejar escritas en el examen:** · "El ancho de banda **nunca** cambia la propagación." · "La distancia **nunca** cambia la transmisión."

1.4 Store-and-forward vs. cut-through

El corazón del Ejercicio 3, mostrado antes y desarrollado después:



El nombre de cut-through: el switch no espera a que la trama quede entera en un buffer, la deja *cuts through*. También se lo llama **cut-through forwarding**.

Lo que cuesta a cambio: si el enlace de salida está ocupado el switch **no puede descartar** (nunca vio el paquete entero), así que **encola antes**, y por eso **propaga la congestión más rápido**: es el *congestion crunch*, y es el motivo por el que los conmutadores de núcleo de las redes de carrier siguen usando store-and-forward.

La tabla que resume la clase entera:

Modelo	términos `d_proc`	términos `L/R`	`Σ d_i/s_i`
Store-and-forward	`k` (uno por conmutador)	`n` (uno por enlace)	`n`
Cut-through	`0` (idealizado)	**1** (se pisan)	`n`

PARTE 2 — PRÁCTICA RESUELTA

> **Cómo se usa esta parte.** Hacé cada ejercicio en papel con la cuenta escrita, sin calculadora al principio. Los ejercicios 1, 2 y 3 son los que tienen que salirte con tinta y sin dudar.

2.1 Ejercicio 1 (guía) — Los cuatro componentes de un paquete en un solo enlace

Datos. Un paquete de **1500 bytes** (el MTU de Ethernet) viaja por un enlace de cobre de **1 Gbps** de **100 km**. El switch tarda **3 µs** en procesar. Sin cola (hipótesis).

a) Calculá los cuatro componentes. **b)** ¿Cuál de los cuatro manda?

Paso 0 — Convertir TODO a las unidades correctas antes de calcular.

```
L = 1500 bytes × 8 bits/byte = 12 000 bits      ⚠ EN BITS
R = 1 Gbps   = 10⁹ bit/s
d = 100 km   × 1000 m/km   = 100 000 m = 10⁵ m  ⚠ EN METROS
s = 2·10⁸ m/s (cobre)
```

Paso 1 — Retardo de transmisión (el tiempo de meter los 12 000 bits en el cable, uno por uno; **no depende de los 100 km**: con un enlace de 1 m sería el mismo):

```
d_trans = L / R = 12 000 bits / 10⁹ bit/s = 1,2·10⁻⁵ s = 12 µs = 0,012 ms
```

Paso 2 — Retardo de propagación (lo que tarda la **señal** en recorrer los 100 km; es el mismo para un paquete de 40 bits que para uno de 15 000):

```
d_prop = d / s = 10⁵ m / 2·10⁸ m/s = 5·10⁻⁴ s = 0,5 ms
```

Paso 3 — Retardo de cola y de procesamiento.

```
d_cola = 0
⚠ Sin dato del enunciado, la hipótesis es 0 y se DECLARA. Si el switch
   transmite 1000 paquetes/s y llegan 1000 paquetes/s, el sistema está EN EL
   LÍMITE: la cola CRECE linealmente y no tiene cota superior. Por eso el
   enunciado tiene que declarar la hipótesis.

d_proc = 3 µs = 0,003 ms   (CRC + leer la cabecera + buscar el puerto de salida)
```

Paso 4 — Total y reparto.

```
d_total = d_proc + d_cola + d_trans + d_prop
        = 3 µs + 0 + 12 µs + 500 µs = 515 µs = 0,515 ms

Componente   valor       % del total
──────────────────────────────────────
d_prop       0,500 ms     97,09 %   ←── DOMINA
d_trans      0,012 ms      2,33 %
d_proc       0,003 ms      0,58 %
d_cola       0             0,00 %
──────────────────────────────────────
TOTAL        0,515 ms    100,00 %
```

b) La respuesta, y la interpretación operativa que la cierra:

> En un enlace de **1 Gbps a 100 km** el retardo lo domina la **propagación** (97 %): el enlace no es lento, es **largo**. Subir `R` de 1 a 10 Gbps baja el total de 0,515 a 0,504 ms (mejora el 2 %); anular `d_proc` gana el 0,6 %. **Ninguna mejora de hardware sirve**: la única palanca que queda es acortar la distancia.

⚠️ Errores típicos del ejercicio 1

- **Usar 1500 en vez de 12 000** (factor 8) o **100 en vez de 100 000** (factor 1000).
- **Inventarse una `d_cola` sin dato:** la hipótesis es 0 y se declara.
- **Sumar `d_proc` una vez por enlace:** con 3 enlaces hay **dos** conmutadores.

2.2 Ejercicio 2 (guía) — Las tres reglas de sensibilidad, con números

Datos. Sin cola, paquete de **1000 bytes**, enlace de **10 Mbps**, **50 km** de cobre, `d_proc = 2 µs`.

```
L = 1000 bytes = 8000 bits      R = 10⁷ bit/s
d = 50 km = 5·10⁴ m            s = 2·10⁸ m/s
d_proc = 2 µs                   d_cola = 0

d_trans = 8000/10⁷ = 8·10⁻⁴ s = 800 µs
d_prop  = 5·10⁴/2·10⁸ = 2,5·10⁻⁴ s = 250 µs
d_total = 2 + 0 + 800 + 250 = 1052 µs = 1,052 ms        ← BASE

Variante a: L = 2000 bytes = 16 000 bits
   d_proc 2 µs (igual) · d_cola 0 (igual) · d_prop 250 µs (IGUAL)
   d_trans = 16 000/10⁷ = 1600 µs        (SE DUPLICA)
   d_total = 2 + 0 + 1600 + 250 = 1852 µs = 1,852 ms    (+76 %)

Variante b: R = 20 Mbps
   d_proc 2 µs (igual) · d_cola 0 (igual) · d_prop 250 µs (IGUAL)
   d_trans = 8000/2·10⁷ = 400 µs         (SE REDUCE A LA MITAD)
   d_total = 2 + 0 + 400 + 250 = 652 µs = 0,652 ms      (−38 %)

Variante c: d = 100 km = 10⁵ m
   d_proc 2 µs (igual) · d_cola 0 (igual) · d_trans 800 µs (IGUAL)
   d_prop  = 10⁵/2·10⁸ = 500 µs          (SE DUPLICA)
   d_total = 2 + 0 + 800 + 500 = 1302 µs = 1,302 ms     (+24 %)
```

La tabla para el examen:

Qué duplico	`d_proc`	`d_cola`	`d_trans`	`d_prop`	`d_total`	Cambio
nada (base)	2 µs	0	800 µs	250 µs	**1052 µs**	—
`L` (1000 → 2000 B)	2 µs	0	**1600 µs**	250 µs	**1852 µs**	**+76 %**
`R` (10 → 20 Mbps)	2 µs	0	**400 µs**	250 µs	**652 µs**	**−38 %**
`d` (50 → 100 km)	2 µs	0	800 µs	**500 µs**	**1302 µs**	**+24 %**

Las tres frases, textuales:

> - **El ancho de banda nunca cambia la propagación.** Duplicar `R` dejó `d_prop` exactamente en 250 µs, porque `d/s` no contiene `R`.
> - **La distancia nunca cambia la transmisión.** Duplicar `d` dejó `d_trans` exactamente en 800 µs, porque `L/R` no contiene `d`.
> - **El tamaño del paquete nunca cambia la propagación.** La señal es la misma por más bits que lleve encima.

Corolario: acá `d_trans` (800 µs) es mayor que `d_prop` (250 µs): es un enlace **corto y lento**, y por eso mejoró `R` y no `d`. En un enlace de backbone, con `d_prop` de 25 ms contra `d_trans` de 1,2 µs, el resultado se da vuelta.

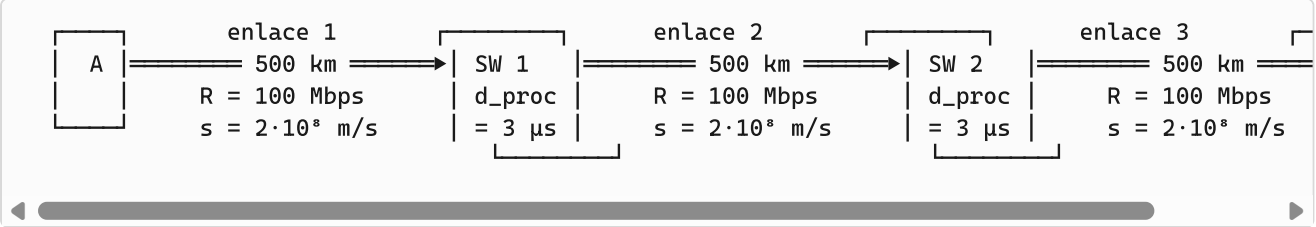
⚠️ Errores típicos del ejercicio 2

- **Meter `R` en la fórmula de propagación:** es el error conceptual más caro del curso (Pregunta 1a del Examen 2009-12, en la Clase 5 §3.1).
- **Creer que "más `R` siempre es mucho mejor":** acá mejoró 38 % solo porque `d_trans` era el 76 % del total. Si la cola fuera el 99 %, no mejoraría nada.

2.3 Ejercicio 3 (★ de examen) — 3 enlaces, 2 conmutadores, con números

Este es el **Examen 2011-03 P5** con números, para verlo antes de la versión simbólica del §3. **Misma estructura, mismo error típico.**

Datos. Paquete de **1500 bytes** de A a B por tres enlaces de cobre unidos por dos conmutadores. Los tres enlaces: **100 Mbps**, **500 km**. `d_proc = 3 µs`. Sin cola.



a) Store-and-forward.

```
Paso 1 — datos:  L = 1500 bytes = 12 000 bits
                 R = 10⁸ bit/s (los tres)  d = 500 km = 5·10⁵ m (los tres)
                 s = 2·10⁸ m/s (los tres)  d_proc = 3 µs POR CONMUTADOR, y hay DOS

Paso 2 — los términos que se repiten
   d_trans (uno) = 12 000/10⁸ = 1,2·10⁻⁴ s = 120 µs        → ×3 =  360 µs
   d_prop  (uno) = 5·10⁵/2·10⁸ = 2,5·10⁻³ s = 2500 µs      → ×3 = 7500 µs
   d_proc        = 3 µs                                      → ×2 =    6 µs   ⚠️ 2, NO 3
   d_cola = 0

Paso 3 — la cuenta, término por término
   d_total = (  3 + 0 + 120 + 2500)     enlace 1
           + (  3 + 0 + 120 + 2500)     enlace 2
           + (  0 + 0 + 120 + 2500)     enlace 3  (el host B no procesa)
           ──────────────────────────────────────────────────
           = 2623 + 2623 + 2620 = 7866 µs = 7,866 ms

Término                   valor      % del total
──────────────────────────────────────────────────
3 × d_propagación         7,500 ms    95,35 %   ←— DOMINA
3 × d_transmisión         0,360 ms     4,58 %
2 × d_procesamiento       0,006 ms     0,08 %
d_cola                    0            0,00 %
──────────────────────────────────────────────────
TOTAL                     7,866 ms   100,00 %
```

b) Cut-through: los conmutadores no almacenan y `d_proc = 0`.

Por qué desaparecen los `L/R` de los enlaces intermedios — el razonamiento central:

```
STORE-AND-FORWARD: el switch 1 tiene que esperar a que ENTRE la trama
                   COMPLETA para mirar la MAC destino y elegir el puerto.
                   Recién ahí arranca por el enlace 2. Como R es el mismo,
                   ese L/R se paga DOS veces: entrar y salir. Idem en el
                   switch 2. De ahí los 3 L/R.

CUT-THROUGH:       el switch ve la MAC destino en los primeros bytes y
                   arranca a emitir por el enlace 2 MIENTRAS sigue
                   recibiendo por el 1. Como R_entrada = R_salida, el
                   ritmo de salida es el de entrada: el switch es
                   TRANSPARENTE.

                   ⇐ El medio se comporta como un solo enlace largo:
                      los tres L/R se PISAN en vez de se SUMAR.
```

La cuenta y la comparación:

```
d = 1·(L/R) + Σ d_i/s_i = 12 000/10⁸ + 3·(5·10⁵/2·10⁸) = 120 + 7500 = 7620 µs

                          store-and-fwd   cut-through (b)
nº de términos L/R                3               1     ←— la diferencia
nº de términos d_proc             2               0
Σ d_i/s_i                         3               3     (igual)
Retardo total                7866 µs         7620 µs
──────────────────────────────────────────────────────────────────────
AHORRO = 7866 − 7620 = 246 µs = 2·(L/R) + 2·d_proc = 240 + 6      ✓

⚠️ Ojo con la proporción: el ahorro es del 3,1 %, NO del 33 %, porque
   acá la propagación (7500 µs) aplasta todo lo demás. Con enlaces muy
   cortos y lentos el ahorro es mucho mayor (ver §2.7).
```

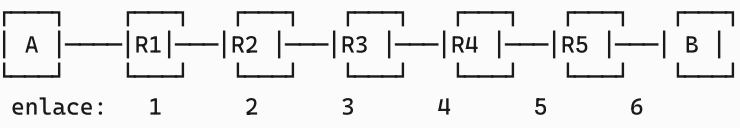
La línea de tiempo, que es la explicación visual del por qué: con store-and-forward, a `t = 120 µs` la trama completa entró al switch 1 (que recién ahí puede mirar la MAC destino y arrancar el enlace 2), a `t = 240 µs` entró completa al switch 2, y a `t = 7866 µs` llega el último bit a B. Con cut-through, a `t = 120 µs` **el último bit ya pasó por el switch 1, por el enlace 2 y por el switch 2**, y llega a B a `t = 7620 µs`. **El "embudo" se vuelve una "tubería": los `L/R` se pisan.**

⚠️ **Y si `R_entrante ≠ R_saliente`, el cut-through se degrada** a store-and-forward: la hipótesis "no hay retardo" SOLO vale si `R_entrada = R_salida` en todos los saltos (ver §2.7).

La moraleja: con cut-through el retardo bajó de 7866 a 7620 µs, un **3,1 %**. Parece poco hasta que uno se da cuenta de que **los 246 µs ahorrados son exactamente los 246 µs de "equipo"**, y que los 7500 µs restantes son **geografía**. **Ningún conmutador, por rápido que sea, puede ganar contra la distancia.**

2.4 Ejercicio 4 — Cinco routers con store-and-forward

Datos. Mensaje de **12 000 bits** de A a B atravesando **cinco routers**, o sea **seis enlaces**. Todos: `R = 1 Mbps`, `d = 100 km`, `s = 2·10⁸ m/s`. `d_proc = 2 µs`.



```
n = 6 enlaces   →  6 términos de d_trans y 6 de d_prop
k = 5 routers   →  5 términos de d_proc y 5 de d_cola     ⚠️ NO 6

Un término de cada tipo:
   d_trans = 12 000/10⁶ = 12·10⁻³ s = 12 ms
   d_prop  = 10⁵/2·10⁸  = 0,5·10⁻³ s = 0,5 ms
   d_proc  = 2 µs = 0,002 ms
   d_cola  = 0 (hipótesis)

Fórmula general:
d_end2end = n·(L/R) + k·d_proc + Σ d_i/s_i
          = 6·(12 000/10⁶) + 5·(2·10⁻⁶) + 6·(10⁵/2·10⁸)
          = 6·0,012 + 0,000010 + 6·0,0005
          = 0,072 + 0,000010 + 0,003
          = 0,075010 s = 75,010 ms

Verificación término por término:
          = 5·(0,002 + 0 + 12 + 0,5) + (0 + 12 + 0,5)
          = 5·12,502 + 12,5 = 62,510 + 12,5 = 75,010 ms     ✓ cuadra
```

Interpretación — y el resultado incómodo:

```
6 × d_transmisión      72,000 ms    95,99 %   ◄── DOMINA
6 × d_propagación       3,000 ms     4,00 %
5 × d_procesamiento     0,010 ms     0,01 %
──────────────────────────────────────────────
TOTAL                   75,010 ms   100,00 %
```

1 Mbps es **muy lento para 100 km**, por eso manda la transmisión: con 100 Mbps, `d_trans = 120 µs` y el total baja a 3,73 ms, un factor 20. Y una observación estructural: los 5 routers aportan 10 µs en total (el 0,013 %): meter un router más y un salto más no cuesta casi nada en procesamiento, lo que cuesta es el `L/R` del enlace extra.

2.5 Ejercicio 5 — *Delay* × *bandwidth*: 8 MB por un enlace de 1 Mbps a 1000 km

Datos. Enlace de **1 Mbps** entre dos puntos a **1000 km** de fibra (`s = 2·10⁸ m/s`). Archivo de **8 MB**. Sin cola, `d_proc` despreciable.

Paso 1 — Latencia de ida y RTT.

```
d = 1000 km = 10⁶ m
d_prop (ida) = 10⁶ / 2·10⁸ = 5·10⁻³ s = 5 ms
RTT ≈ 2 × 5 ms = 10 ms = 0,010 s
   (el "≈" es porque el camino de vuelta puede ser otro, y porque hay
    d_trans y d_proc que no son cero)
```

Paso 2 — Tiempo del archivo entero.

```
8 MB = 8·10⁶ bytes (convención decimal; con MiB serían 8 388 608 B)
En bits: 8·10⁶ × 8 = 6,4·10⁷ bits = 64 000 000 bits
R = 1 Mbps = 10⁶ bit/s
t = L/R = 6,4·10⁷ / 10⁶ = 64 s
```

Interpretación: la propagación es irrelevante acá: el archivo se está mandando durante 64 s, no durante 10 ms.

Paso 3 — Los bytes en vuelo: el *bandwidth-delay product*.

```
BDP = R · RTT = 10⁶ bit/s × 0,010 s = 10⁴ bits = 10 000 bits
En bytes: 10 000/8 = 1250 bytes

 A ═══════════════════════════════════════════ B
   ◄──────── 1250 bytes como máximo ────────►

 En los 10 ms que tarda el ACK en volver, salen 10⁶ × 0,010 = 10 000
 bits = 1250 bytes. Si la ventana es menor, el emisor ESPERA un ACK y el
 enlace queda VACÍO ese tiempo.
```

Paso 4 — Tres ventanas concretas (`throughput = ventana / RTT`):

Ventana	¿vs. BDP?	Throughput	Tiempo para 8 MB
500 B	< BDP	`500/0,010` = 50 000 B/s = **0,40 Mbps**	`8·10⁶/50 000` = **160,0 s**
1000 B	< BDP	`1000/0,010` = 100 000 B/s = **0,80 Mbps**	`8·10⁶/100 000` = **80,0 s**
1250 B	= BDP	`1250/0,010` = 125 000 B/s = **1,00 Mbps**	**64,0 s** ✓
2000 B	> BDP	`min(R, v/RTT)` = **1 Mbps** (el enlace es el techo)	**64,0 s**

La respuesta a (c):

> La ventana **menor que el BDP** limita el throughput a `ventana/RTT` y el enlace nunca se llena. La regla exacta es `ventana ≥ BDP = R · RTT`. Por eso existe `RFC 7323` (*Window Scale*, 2014): sin él el máximo son 65 535 bytes, y en un enlace de 100 Mbps con 12 ms de RTT eso da apenas `65 535/0,012 = 52,4 Mbps`. Y para enlaces largos y rápidos el cálculo es brutal: 100 Gbps con 100 ms de RTT da `BDP = 10¹⁰ bits = 1,25 GB`, y ninguna ventana TCP del mundo llena eso.

2.6 Ejercicio 6 — RTT de ida y vuelta, latencia y jitter

Datos. Tres paquetes del mismo flujo de audio salen con 20 ms de separación. Sus RTT son **80 ms**, **95 ms** y **110 ms**.

Paso 1 — Los conceptos. Latencia = el tiempo de **un viaje de ida**; ≈ `RTT/2`, y solo si el camino de vuelta es el mismo que el de ida. **RTT** = ida y vuelta, lo que mide `ping`. **Jitter** = la **variación** del retardo entre paquetes del mismo flujo; no es el retardo, es cuánto cambia.

Paso 2 — Los números.

```
Serie de RTT: 80 ms, 95 ms, 110 ms

RETARDO MEDIO:  RTT_promedio = (80+95+110)/3 = 285/3 = 95 ms
                latencia ≈ 95/2 = 47,5 ms

JITTER, tres métricas:

(i)   Pico a pico (la que usa el jitter buffer):
            J = 110 − 80 = 30 ms

(ii)  Desviación estándar:
            σ = √( (Σ (R_i − R_prom)²) / n )
              = √( (80−95)² + (95−95)² + (110−95)² ) / 3 )
              = √( (−15)² + 0² + 15² ) / 3 = √( (225 + 0 + 225) / 3 )
              = √( 450/3 ) = √150 = 12,25 ms

(iii) Variación media absoluta:
            J = ( |80−95| + |95−95| + |110−95| ) / 3 = (15+0+15)/3 = 10 ms
```

Paso 3 — Qué le pasa al audio, y por qué 30 ms de jitter es un problema real. El códec reproduce una muestra cada 20 ms; los paquetes salen en `t = 0, 20, 40` y el reproductor tiene un retardo base de 60 ms:

```
P1 sale en t=  0, tarda  80 ms → llega en t=  80 (lo esperaba en t=  60)
P2 sale en t= 20, tarda 110 ms → llega en t= 130 (lo esperaba en t=  80)
P3 sale en t= 40, tarda  95 ms → llega en t= 135 (lo esperaba en t= 100)

■ SILENCIO ■ → CLIC en el audio a los 80, 100 y 120 ms
```

⚠️ **El problema NO es que tarden: es que lleguen a ráfagas.** Con los tres RTT iguales a 95 ms, el retardo sería de 95 ms (el doble que una llamada telefónica) y el audio sería **perfecto**. ⇒ **un retardo alto y constante es mejor que uno medio y variable.** Por eso existe el jitter buffer: mete un retardo **fijo** (30-40 ms), reordena lo que llegó tarde y rellena los huecos. El compromiso: buffer grande = más tolerancia al jitter, más latencia.

Paso 4 — La definición con las palabras del Examen 2010-02 P5.

> **Solución oficial:** *"El retardo es el tiempo que tarda un paquete desde que sale del emisor hasta que llega al receptor, mientras que el jitter es la variación de este retardo entre distintos paquetes de un mismo flujo."* Y el remate: *"Si todos los paquetes del flujo siguieran el mismo camino pero no hubiera congestión, podríamos mantener el retardo constante (aunque sea de varios segundos) mientras que el jitter sería mínimo o nulo."*

Paso 5 — La tabla de sensibilidad, que es la parte b) de esa pregunta:

Escenario	retardo	jitter	¿audio?
Enlace de 5 ms, sin congestión	10 ms	0 ms	PERFECTO
Enlace de 300 ms, sin congestión	600 ms	0 ms	molesto, tolerable
Enlace de 30 ms, con colas variables	45 ms	30 ms	CLICS
Enlace de 300 ms con ECMP y 2 caminos	150 ms	12 ms	aceptable

Paso 6 — De dónde sale el jitter: de la **cola** de un router (la fuente principal: todo el jitter es `d_cola` variable), de una **pérdida con retransmisión** (el hueco dura un RTO, mínimo 200 ms en Linux), del **ECMP** y de los **caminos asimétricos**.

⚠️ Errores típicos del ejercicio 6

- **Decir "el jitter es el retardo":** no, es **la variación**. Con tres RTT de 90 ms el jitter es 0 y el retardo es 90 ms.
- **Calcular "latencia = RTT/2" sin la salvedad:** solo vale con camino simétrico (lo dice la oficial de 2011-03 P3).

⚠️ Actualización a 2026 — qué pasó con el retardo

Sigue igual — de memoria: - Los **cuatro componentes**, sus fórmulas y sus unidades. - `d_trans = L/R` con `L` en **bits**; `d_prop = d/s` con `d` en **metros**. - `d_cola` es el único componente **variable** y el único que refleja la **congestión**. - **Jitter = variación del retardo**, y viene de la `d_cola`. - **Latencia ≈ RTT/2** solo con camino simétrico.

Cambió o hay que reencuadrar: - **La `d_cola` pasó de ser un accidente a ser el objetivo de diseño.** El *bufferbloat* es hoy el problema dominante de las redes de acceso. Linux usa **FQ-CoDel** por defecto desde el kernel 3.5 (2012), que descarta **temprano** para que el retardo no explote. - **El objetivo del control de congestión pasó de "evitar la pérdida" a "evitar la cola".** `RFC 9438` (CUBIC, *Standards Track* 2023) sigue reduciendo la ventana por pérdida, pero **ECN** (`RFC 3168`) y **L4S** (`RFC 9331`, 2023) marcan con umbral bajo para conseguir latencia mínima sin descartes. - **El BDP se volvió imposible de llenar.** Un enlace continental de 100 Gbps con 100 ms de RTT tiene un BDP de **1,25 GB**. `RFC 7323` (*Window Scale*) multiplicó la ventana pero ni así alcanza: por eso **BBR** (`draft-ietf-ccwg-bbr`, *Experimental*, **nunca fue RFC**) no usa una ventana sino un modelo de *bottleneck bandwidth* × *propagation delay*. - **`d_proc` dejó de ser despreciable en algunos casos.** En un router de 100/400 Gbps la CPU corre MPLS, ACLs, ECMP y QoS en el plano de datos: `d_proc` sube a **1-10 µs**, del orden del `d_trans`. **Ojo:** lo que sigue siendo cierto es que **no depende de la carga**, y por eso no es el componente de la congestión. - **La latencia mejoró muy poco:** la fibra de nueva generación (ZOW, *zero water mark*) baja el retardo de los enlaces continentales de ~5 a ~3,5 ms por latitud, pero no lo abolió. **Ninguna mejora de hardware va contra la `d/s`.**

> **Consecuencia para el examen:** las **fórmulas y el análisis por componentes no cambiaron en 37 años**. Resolvé con los números del enunciado y, si hay lugar, sumá una línea de que hoy el objetivo es **evitar la cola**.

2.7 Ejercicio 7 — Store-and-forward vs. cut-through, con números que se ven

Datos. Paquete de **10 000 bits** (1250 bytes) por **4 enlaces** de **10 Mbps** y **20 km** unidos por **3 conmutadores**. `d_proc = 4 µs`. `s = 2·10⁸ m/s` en los cuatro.

```
Un término de cada tipo:
   d_trans = 10 000/10⁷ = 10⁻³ s = 1 ms
   d_prop  = 2·10⁴/2·10⁸  = 10⁻⁴ s = 0,1 ms
   d_proc  = 4 µs = 0,004 ms

a) STORE-AND-FORWARD
   d = 3·(0,004) + 4·(1) + 4·(0,1) = 0,012 + 4 + 0,4 = 4,412 ms

b) CUT-THROUGH con d_proc real
   d = 3·(0,004) + 1·(1) + 4·(0,1) = 0,012 + 1 + 0,4 = 1,412 ms

c) CUT-THROUGH idealizado
   d = 0 + 1·(1) + 4·(0,1) = 1,4 ms

Modelo                 a) S&F     b) CT real   c) CT ideal
──────────────────────────────────────────────────────────
términos de L/R           4           1            1
términos de d_proc        3           3            0
términos de d_prop        4           4            4
──────────────────────────────────────────────────────────
RETARDO (ms)           4,412        1,412        1,400

Ahorro (a → b) = 3,000 ms = (n−1)·(L/R) = 3·1 ms              ✓
Ahorro (b → c) = 0,012 ms (el d_proc de los 3 switches)
```

El cut-through ahorró el 68 % del retardo total — al revés que en el §2.3. La razón es que acá `d_trans` (1 ms) domina `d_prop` (0,1 ms) por 10 a 1, mientras que en el §2.3 `d_prop` (2,5 ms) domina `d_trans` (0,12 ms) por 20 a 1, y en una LAN los dos son despreciables frente a `d_proc`. **No hay respuesta universal: hay que calcular.**

El caso patológico: `R` distinto en cada enlace. Con $R_1 = 100$ Mbps, $R_2 = 10$ Mbps y $R_3 = 100$ Mbps, el switch 1 recibe a 100 Mbps y tiene que emitir a 10 Mbps: **se acumula cola en el switch 1**, cada bit que entra tarda 10 veces más en salir, y el switch se comporta exactamente como store-and-forward con $10 \cdot (L/R) = 10$ ms de espera. **Peor que store-and-forward**, y además propaga la congestión antes de tiempo.

2.8 Ejercicio 8 — Satélite contra fibra: por qué ancho de banda no es latencia

```
Opción A (satélite):  36 000 km por aire,  s = 3·10⁸ m/s,  R = 2 Mbps
Opción B (fibra):      1000 km de fibra,   s = 2·10⁸ m/s,  R = 1 Gbps
L = 12 000 bits, d_proc = 5 µs (un conmutador), sin cola

A) SATÉLITE
   d_trans = 12 000/2·10⁶ = 6 ms
   d_prop  = 3,6·10⁷/3·10⁸ = 0,12 s = 120 ms
   d_proc  = 5 µs = 0,005 ms
   d_total = 0,005 + 6 + 120 = 126,005 ms

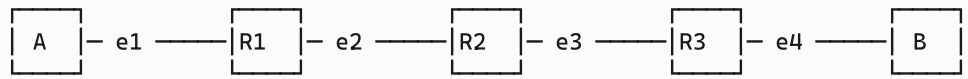
B) FIBRA
   d_trans = 12 000/10⁹ = 12 µs = 0,012 ms
   d_prop  = 10⁶/2·10⁸ = 5 ms
   d_total = 0,005 + 0,012 + 5 = 5,017 ms
```

	A: satélite	B: fibra
`R`	2 Mbps	1 Gbps → 500× más ancho
distancia	36 000 km	1 000 km → 36× más lejos el satélite
`d_trans`	6,000 ms	0,012 ms → 500× peor el satélite
`d_prop`	120,000 ms	5,000 ms → 24× peor el satélite
TOTAL	**126,005 ms**	**5,017 ms** → 25× peor el satélite
latencia / RTT / BDP	63 ms / 252 ms / 63 kB	2,5 ms / 11 ms / 1 375 kB

La lectura al revés, que es el error conceptual del curso:

> "El satélite tiene 2 Mbps y la fibra 1 Gbps, o sea que la fibra es 500 veces más rápida." **FALSO.** Lo correcto: el satélite tiene **500 veces menos ancho de banda** y aun así la llamada tarda **25 veces más**. Porque la latencia no la manda el ancho de banda: la mandan la **distancia** y la **velocidad de propagación**.
>
> Un geoestacionario está a 36 000 km: la luz tarda 120 ms en bajar y 120 ms en volver. Por eso el satélite **sirve para streaming** (tolera 2 s de retardo) y **no para VoIP interactivo** (no tolera más de 150 ms de ida). **Éste es el mejor ejemplo del curso de por qué ancho de banda y latencia son dos cosas distintas.**
>
> La alternativa son los satélites de **órbita baja** (LEO): a 550 km de altura la propagación de ida es $550\,000/3{\cdot}10^8$ = 1,8 ms, y las constelaciones de cientos de satélites llegan a 20-50 ms. El problema se muda a la conmutación entre satélites. ⚠️ Verificar las cifras de despliegue: cambian mes a mes.

2.9 Ejercicio 9 — Tres enlaces heterogéneos, cuenta larga




```
e1: R =   1 Mbps, d =   2 km, cobre        e3: R =  10 Mbps, d =  50 km, cobre
e2: R = 100 Mbps, d = 300 km, fibra        e4: R =   1 Gbps, d =  80 km, fibra
L = 1000 bytes = 8000 bits    d_proc = 3 µs por router (3 routers)    sin cola
```

Cada término escrito:

```
enlace  R        d        d_trans = L/R    d_prop = d/s    d_proc   subtotal
──────────────────────────────────────────────────────────────────────────
e1      10⁶      2·10³ m  8000/10⁶=8,000  2·10³/2·10⁸=0,010  0,003   8,013 ms
e2      10⁸      3·10⁵ m  8000/10⁸=0,080  3·10⁵/2·10⁸=1,500  0,003   1,583 ms
e3      10⁷      5·10⁴ m  8000/10⁷=0,800  5·10⁴/2·10⁸=0,250  0,003   1,053 ms
e4      10⁹      8·10⁴ m  8000/10⁹=0,008  8·10⁴/2·10⁸=0,400    −      0,408 ms
──────────────────────────────────────────────────────────────────────────
TOTAL d_end2end = 8,013 + 1,583 + 1,053 + 0,408 = 11,057 ms

Por tipo:   d_trans 8,888 ms (80,4 %) · d_prop 2,160 ms (19,5 %) · d_proc 0,009 ms
Por enlace: e1 = 72,5 % · e2 = 14,3 % · e3 = 9,5 % · e4 = 3,7 %
```

La pregunta que hay que responder, y el plan de mejora:

```
El enlace e1 aporta el 72,5 % del retardo, y es el ÚNICO de los cuatro que está
a menos de 100 km. Porque es el ÚNICO lento.

⚠️ EL CUELLO DE BOTELLA NO ES EL MÁS LARTO NI EL MÁS CARO:
   ES EL DE MENOR d_trans.

Plan, por peso invertido: subir e1 a 10 Mbps (d_trans = 0,800 ms) baja el total a
0,813 + 1,583 + 1,053 + 0,408 = 3,857 ms, o sea un 65 % de mejora con un solo cambio.
Recién ahí e2 a 1 Gbps → 2,857 ms, y e3 a 100 Mbps → 2,257 ms.
```

2.10 Ejercicio 10 — Comparar dos topologías con la misma distancia total

```
RUTA:      3 saltos de 1000 km a 10 Mbps, 2 routers, d_proc = 5 µs
DIRECTA:   1 salto de 3000 km a 1 Gbps,   0 routers
L = 12 000 bits, fibra (s = 2·10⁸), sin cola

RUTA:      d_trans = 12 000/10⁷ = 1,2 ms ×3 = 3,6 ms
           d_prop  = 10⁶/2·10⁸ = 5 ms ×3 = 15 ms
           d_proc  = 5 µs ×2 = 0,010 ms
           d_total = 3,6 + 15 + 0,010 = 18,610 ms

DIRECTA:   d_trans = 12 000/10⁹ = 0,012 ms ×1
           d_prop  = 3·10⁶/2·10⁸ = 15 ms ×1
           d_total = 0,012 + 15 + 0 = 15,012 ms
```

	RUTA (3×1000 km, 10 Mbps)	DIRECTA (3000 km, 1 Gbps)
nº de enlaces · distancia total	3 · 3000 km	1 · 3000 km → **igual distancia**
`d_trans`	3,600 ms	0,012 ms
`d_prop`	15,000 ms	15,000 ms → **igual**
TOTAL	**18,610 ms**	**15,012 ms**

Y el inverso, que es el que más rinde: una directa de **1500 km a 1 Gbps** da `d_prop = 7,5 ms` y `d_total = 7,512 ms`; una ruta de **3 saltos de 500 km a 10 Mbps** da `d_prop = 7,5 ms` (igual) y `d_trans = 3,6 ms`, o sea `d_total = 11,110 ms`.

> ⚠️ A **igual distancia total**, la ruta corta gana 3,6 ms. **La distancia se reparte; la transmisión se suma.** Esa es la frase del ejercicio.

Agregar un **router** cuesta casi nada (5 µs de `d_proc`); agregar un **enlace** cuesta `L/R + d/s`. Por eso las redes de backbone tienen pocos saltos y muy rápidos: cada salto extra es un `L/R` extra que se paga en todos los paquetes.

2.11 Ejercicio 11 — *Delay* × *bandwidth* de una descarga grande

Datos. Bajás **100 MB** por un enlace de **5 Mbps** con **RTT = 200 ms**. Sin cola.

```
BDP = 5·10⁶ bit/s × 0,200 s = 10⁶ bits = 125 000 bytes = 125 kB

a) VENTANA 16 KB:  16 384 < 125 000 → SÍ limita
   Throughput = 16 384/0,200 = 81 920 B/s = 0,655 Mbps
   Tiempo     = 100·10⁶/81 920 = 1220,7 s = 20,3 minutos
b) VENTANA 1 MB:   1 048 576 > 125 000 → sobra
   Throughput = 1 048 576/0,200 = 41,9 Mbps → el tope es R = 5 Mbps
   Tiempo = 100·10⁶/(5·10⁶/8) = 100·10⁶/625 000 = 160 s = 2,67 minutos
c) MÁXIMO: 5 Mbps, alcanzado cuando W ≥ 125 kB (el BDP).
   8 KB → 0,33 Mbps → 40,5 min  ·  64 KB → 2,62 Mbps → 5,1 min  ·  125 KB → 5 Mbps
```

La lección: el enlace **no es lento** —a 5 Mbps los 100 MB tardarían 160 s siempre—, lo que tarda más es la **ventana**. Con 8 KB (el `rwnd` por defecto de Windows durante años) tardás 40,5 minutos en vez de 2,67: un factor 15. `RFC 7323` (*Window Scale*, 2014) es lo que cubre el BDP: sin él el máximo son 65 535 bytes, y acá eso da el 52 %.

2.12 Ejercicio 12 — Sensibilidad de punta a punta sobre el problema de examen

Datos. Los del Ejercicio 3: `L = 1500 bytes`, tres enlaces de **100 Mbps** y **500 km**, dos conmutadores de `d_proc = 3 µs`, store-and-forward. **Base:** `d_total = 360 + 6 + 7500 = 7866 µs = 7,866 ms`.

```
a) R de 100 a 200 Mbps
   d_trans = 12 000/2·10⁸ = 60 µs ; ×3 = 180 µs   (SE REDUCE)
   d_prop y d_proc no cambian → 180 + 6 + 7500 = 7686 µs = 7,686 ms
   Variación: −180 µs = −2,3 %

b) d de 500 a 1000 km
   d_trans no cambia (360 µs) ; d_proc no cambia (6 µs)
   d_prop = 10⁶/2·10⁸ = 5 ms = 5000 µs ; ×3 = 15 000 µs
   d_total = 360 + 6 + 15 000 = 15 366 µs = 15,366 ms
   Variación: +7500 µs = +95,4 %

c) R = 1 Gbps  Y  d = 100 km  (las dos cosas juntas)
   d_trans = 12 000/10⁹ = 12 µs ; ×3 =  36 µs
   d_prop  = 10⁵/2·10⁸ = 500 µs ; ×3 = 1500 µs
   d_proc  = 3 µs ; ×2 = 6 µs
   d_total = 36 + 6 + 1500 = 1542 µs = 1,542 ms
```

Escenario	`R`	`d`	`d_total`	vs. base
base	100 Mbps	500 km	**7,866 ms**	—
a) `R` ×2	200 Mbps	500 km	**7,686 ms**	**−2,3 %**
b) `d` ×2	100 Mbps	1000 km	**15,366 ms**	**+95,4 %**
c) `R` ×10 y `d` ÷5	1 Gbps	100 km	**1,542 ms**	**−80,4 %**

La interpretación, que es lo que cierra los 10 puntos del problema:

> - **b)** Duplicar la distancia casi duplica el retardo, porque `d_propagación` es el 95 % del total.
> - **a)** Duplicar el ancho de banda mejora apenas (2,3 %), porque `d_trans` es el 4,6 %. **No tocó `d_prop`.**
> - **c)** Con las dos juntas baja un 80 %, pero `d_trans` bajó ×10 (360 → 36 µs, porque `R` ×10) y `d_prop` bajó ×5 (7500 → 1500 µs, porque `d` ÷5). `d_prop` **no bajó ×10**: depende de la distancia, no del ancho de banda.
>
> **La frase que hay que dejar escrita:** *"En una WAN larga el retardo lo domina la propagación y la palanca es la distancia y el medio. En un enlace corto y lento lo domina la transmisión y la palanca es el ancho de banda. En una red saturada lo domina la cola y la única palanca es reducir el tráfico que llega."*

⚠️ Errores típicos de los ejercicios 7 a 12

- **Sumar los `d_proc` por enlace:** siempre son `n − 1` en una línea.
- **Aplicar el mismo `R` a todos los enlaces:** cada `d_trans` usa el `R` **de su enlace**.
- **Usar un solo `s` sin mirar el medio:** si uno es satélite, ese usa $3 \cdot 10^8$.
- **Comparar BDP en bits contra ventana en bytes:** hay que convertir.
- **Confundir throughput con retardo:** bajar un archivo más rápido es una cosa; que llegue antes es otra.

PARTE 3 — EL PROBLEMA DE EXAMEN

3.1 ⭐ ⭐ Examen 2011-03, Pregunta 5 (10 puntos) — retardo end-to-end

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011, Pregunta 5 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Este es EL problema de retardos del curso.** Aparece reformulado en el Simulacro 1 (Clase 7).

Enunciado (texto literal del examen)

> Considere un paquete de longitud L que tiene su origen en el sistema terminal A y que viaja a través de tres enlaces hasta un sistema terminal de destino B. Estos tres enlaces están conectados mediante dos dispositivos de conmutación de paquetes. Sean `d_i`, `s_i` y `R_i` la longitud, la velocidad de propagación y la velocidad de transmisión del enlace i, para `i = 1, 2, 3`. Cada dispositivo de conmutación de paquetes retarda cada paquete `d_proc`.
>
> **a)** Suponiendo que no se produce retardo de cola, ¿cuál es el retardo total terminal a terminal del paquete en función de `d_proc`, `d_i`, `s_i`, `R_i` (`i = 1, 2, 3`) y L?
>
> **b)** Suponga ahora que `R_1 = R_2 = R_3 = R` y `d_proc = 0`. Suponga también que el conmutador de paquetes no almacena los paquetes y los reenvía, sino que transmite inmediatamente cada bit que recibe sin esperar a que llegue el paquete completo. ¿Cuál será el retardo terminal a terminal?

Solución oficial (completa, textual)

> **a)** *"En general el retardo extremo a extremo es la suma de los componentes del retardo: procesamiento, cola, transmisión y propagación. Si despreciamos el retardo de cola, el retardo de extremo a extremo se expresa como:*
>
> `d_end2end = Σ_{i=1..3} d_proc + L/R_i + d_i/s_i` *
>
> *"Para el caso concreto, se reduce a:* `d_end2end = d_proc + d_proc + L/R1 + L/R2 + L/R3 + d1/s1 + d2/s2 + d3/s3` "
>
> **b)** *"Dado que los bits se transmiten inmediatamente, los conmutadores de paquetes no introducen ningún retardo (en particular no introducen retardo de transmisión), y por lo tanto ahora el retardo de extremo a extremo se expresa como:* `d_end2end = L/R + d1/s1 + d2/s2 + d3/s3` "

Análisis de la solución oficial

Lo que está muy bien: 1. **El patrón correcto en el apartado a):** el `d_proc` aparece **dos veces**, escrito literalmente `d_proc + d_proc`. **Ese es el 80 % de los puntos**, y muchísimos alumnos lo escriben tres

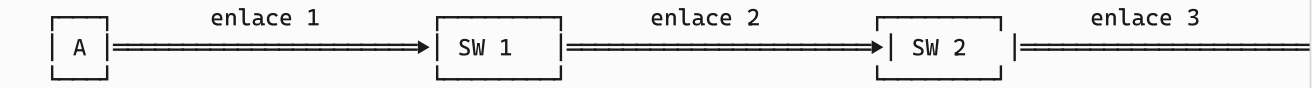
veces. 2. **Los tres `L/R_i` con el `R_i` de cada enlace** (el apartado a) NO dice que sean iguales), y los tres `d_i/s_i` con su `s_i`. 3. **En el apartado b) justifica con una frase**, y no solo con el número: "*los conmutadores no introducen ningún retardo, y en particular no introducen retardo de transmisión*". Y ya usa `R` único, que es la hipótesis nueva del enunciado.

Lo que falta, y son los puntos que se ganan: 1. **No explica por qué `d_proc` va dos veces.** Lo escribe bien pero no lo **argumenta**. 2. **No nombra el modelo** del apartado b): lo describe pero no lo llama **cut-through forwarding**. 3. **No cuantifica la mejora.** "Cuál será el retardo" pide un número, y decir "es menor" sin cuánto deja puntos. 4. **No idealiza con advertencia** (en un switch real hay que leer la MAC destino, ≈14 bytes) ni dice qué pasa si `R_entrada ≠ R_salida`.

Resolución completa — para los 10 puntos

Parte a)

Paso 1 — Contar los términos antes de escribir nada.




```
n = 3 enlaces      →  3 términos de d_transmisión  +  3 de d_propagación
k = 2 conmutadores →  2 términos de d_procesamiento +  2 de d_cola
cola               →  0 por hipótesis del enunciado
```

Paso 2 — Fórmula general y concretada.

```
GENERAL:
    d_end2end = Σ_{i=1..3} [ d_proc + d_cola + L/R_i + d_i/s_i ]

CON d_cola = 0:

⚠️ El sumatorio trae 3·d_proc, pero hay que descontar el de los extremos:
   A y B son SISTEMAS TERMINALES, no dispositivos de conmutación.

CONCRETA:
d_end2end =  d_proc + d_proc                          ← 2 conmutadores
           + L/R₁ + L/R₂ + L/R₃                       ← 3 enlaces
           + d₁/s₁ + d₂/s₂ + d₃/s₃                    ← 3 enlaces

         =  2 · d_proc + Σ_{i=1..3} L/R_i + Σ_{i=1..3} d_i/s_i
```

Paso 3 — La frase que más suma, porque es lo que demuestra que lo entendiste.

> El retardo de transmisión y el de propagación aparecen **una vez por enlace**, y son tres enlaces: hay **tres términos de cada uno**. El retardo de procesamiento aparece **una vez por dispositivo de conmutación**, y hay **dos**: hay **dos términos de `d_proc`**. Los sistemas terminales A y B no son dispositivos de conmutación y no aportan `d_proc`.

Paso 4 — Forma alternativa compacta, y el caso homogéneo.

$d_{end2end} = 2 \cdot d_{proc} + L \cdot (1/R_1 + 1/R_2 + 1/R_3) + (d_1/s_1 + d_2/s_2 + d_3/s_3)$. Y si los tres enlaces son homogéneos: $d_{end2end} = 2 \cdot d_{proc} + 3 \cdot (L/R) + 3 \cdot (d/s)$.

> ⚠️ **Nota sobre la oficial:** escribe literalmente `d_proc + d_proc + L/R1 + L/R2 + L/R3 + d1/s1 + d2/s2 + d3/s3`, que es **exactamente** lo mío. **No hay diferencia con la oficial.** La oficial simplemente no explicita la fórmula compacta ni la justificación del "dos".

Parte b)

Paso 1 — Qué cambia exactamente.

```
El enunciado agrega DOS hipótesis numéricas:
    (i)   R₁ = R₂ = R₃ = R      → los tres L/R se vuelven uno solo
    (ii)  d_proc = 0              → desaparece el procesamiento

Y UNA hipótesis de comportamiento:
    (iii) el switch NO almacena: transmite cada bit apenas lo recibe
```

Paso 2 — Por qué desaparecen los L/R de los enlaces intermedios.

```
STORE-AND-FORWARD:  el switch 1 tiene que esperar a que ENTRE la trama
                    COMPLETA para mirar la MAC destino y elegir el puerto.
                    Recién ahí arranca por el enlace 2. Como R es el mismo,
                    ese L/R se paga dos veces: entrar y salir. Idem en el
                    switch 2. De ahí los 3 L/R.

CUT-THROUGH:        el switch ve la MAC destino en los primeros bytes y
                    arranca a emitir por el enlace 2 MIENTRAS sigue
                    recibiendo por el 1. Como R_entrada = R_salida, el
                    ritmo de salida es el de entrada: el switch es
                    TRANSPARENTE, no acumula nada.

                    ⇐ El medio se comporta como un solo enlace largo:
                       los tres L/R se PISAN en vez de se SUMAR.

Detalle fino: al switch le hacen falta los primeros bytes para conocer el puerto de
salida, así que transmite con un atraso de (14·8)/R = 1,12 µs, no de forma instantánea.
```

Paso 3 — La cuenta y la comparación con el apartado a).

```
d_end2end = 1 · (L/R) + Σ_{i=1..3} (d_i/s_i) = L/R + d₁/s₁ + d₂/s₂ + d₃/s₃

                        (a) store-and-fwd    (b) cut-through
──────────────────────────────────────────────────────────────
términos de L/R                 3                  1
términos de d_proc              2                  0
términos de d_i/s_i             3                  3   (igual)
──────────────────────────────────────────────────────────────
RESULTADO           2·d_proc + Σ L/R_i + Σ d_i/s_i    (L/R) + Σ d_i/s_i
──────────────────────────────────────────────────────────────
DIFERENCIA      MEJOR EN (b) EN: 2·d_proc + 2·(L/R)
```

El ahorro es exactamente $(n - 1) \cdot (L/R) + k \cdot d_{proc} = 2 \cdot (L/R) + 2 \cdot d_{proc}$.

Paso 4 — El nombre del modelo y la advertencia realista.

> **El modelo del apartado b) se llama `cut-through forwarding` (reenvío al vuelo).** La trama *cuts through* el conmutador: no espera a quedar entera en un buffer.
>
> **Advertencia realista:** un conmutador real **no puede** reenviar el primer bit, porque la dirección MAC destino está en los **6 primeros bytes** de la trama, precedidos por 8 de preámbulo. Tiene que leer unos **14 bytes** antes de poder decidir: `(14 · 8)/R` segundos. A 100 Mbps son **1,12 µs**, y un `d_proc` real de switch cut-through está entre 1 y 2 µs, no 0. La solución oficial pone cero porque **el enunciado fija esa hipótesis**, y está bien que lo haga: lo que se corrige es lo que dice el enunciado. **Señalar el matiz suma puntos.**

Paso 5 — La observación profunda, si queda lugar. El resultado del apartado b) **supone que** `R_1 = R_2 = R_3 = R`. Si el conmutador recibiera más rápido de lo que transmite (`R_1 = 1 Gbps`, `R_2 = 100 Mbps`), se acumularía cola y **se comportaría como un store-and-forward**: el bit que entra primero saldría último. El cut-through sin `R` uniforme **agrega congestión antes de tiempo** (*congestion crunch*), y ésa es la razón real por la que los conmutadores de núcleo de las redes de carrier no lo usan.

Paso 6 — La tabla resumen, que es la respuesta "de 10":

Modelo	`d_end2end`	Cuándo
Store-and-forward (a)	`2·d_proc + Σ L/R_i + Σ d_i/s_i`	el caso general
Cut-through (b)	`1·(L/R) + Σ d_i/s_i`	latencia crítica
Store-and-forward con `R` y `d_proc` uniformes	`0 + 1·(L/R) + Σ d_i/s_i`	el "mejor caso", **imposible**

> **Nota: la solución oficial coincide exactamente con mi resolución en los dos apartados.** No hay divergencia. Lo que agregué es la justificación escrita del "dos `d_proc`", el nombre del modelo, la cuantificación del ahorro y las advertencias realistas. Eso, en un examen de 10 puntos, es la diferencia entre un 6 y un 10.

3.2 Recordatorio — dónde están los otros problemas de retardo

Estos **ya están resueltos enteros en la Clase 5** y no se repiten: `2009-12` **P1** (10 pts, componentes del retardo, ancho de banda vs. velocidad de propagación y congestión → Clase 5 §3.1), `2011-03` **P2** (6 pts, congestión, pérdidas y retardo → Clase 5 §3.2) y `2010-02` **P5a** (8 pts, jitter vs. retardo → Clase 5 §1.5; los números en el Ejercicio 6 de esta clase). Los tres van con la solución oficial textual y su análisis línea por línea.

PARTE 4 — CIERRE

📌 Resumen

1. `d_end2end = Σ (d_proc + d_cola + L/R + d/s)`, y el total se **suma por cada enlace** y por cada router.
2. **Con `n` enlaces hay `n` términos de transmisión y `n` de propagación, pero solo `n − 1` de procesamiento.** Por eso la oficial del 2011-03 P5 escribe `d_proc + d_proc`.
3. `d_trans = L/R`, con `L` en **bits**: **no depende del medio ni de la distancia**.
4. `d_prop = d/s`, con `d` en **metros**: **no depende del paquete ni del ancho de banda**.
5. `d_cola` **es el único componente variable** y el **único que refleja la congestión**.
6. **Las tres reglas de sensibilidad:** duplicar `L` cambia solo `d_trans`; duplicar `R` cambia solo `d_trans` (a la mitad); duplicar `d` cambia solo `d_prop`.
7. **Store-and-forward = `n` términos `L/R`; cut-through = 1**, porque los `L/R` se pisan en vez de sumando. El ahorro es `(n − 1)·(L/R) + k·d_proc`.
8. **Cut-through solo es gratis si `R_entrada = R_salida` en todos los saltos**, y exige leer la MAC destino (≈14 bytes), así que el `d_proc = 0` del enunciado es una idealización.
9. `BDP = R · RTT` son los bits en vuelo; si `ventana < BDP`, el throughput queda limitado por la ventana y no por el enlace.
10. **Latencia ≈ RTT/2 solo con camino simétrico; jitter = variación del retardo**, y **rompe el audio mucho antes que la latencia**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un paquete de `L` bits cruza 3 enlaces unidos por 2 conmutadores, con `d_proc` por conmutador y sin cola. Escribí la fórmula completa y decí cuántas veces aparece cada término.

<details>
<summary>Respuesta</summary>
</details>

2. En el mismo problema, ahora con cut-through, `R_1 = R_2 = R_3 = R` y `d_proc = 0`. ¿Cuál es el retardo y por qué mejora?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué un switch cut-through real NO puede tener retardo cero?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Qué componente domina el retardo, y por qué no hay una respuesta universal?

<details>
<summary>Respuesta</summary>
</details>

5. Un enlace de 20 Mbps une Montevideo y Buenos Aires, y el RTT medido es 80 ms. ¿Cuántos bytes pueden estar en vuelo? ¿Qué pasa si la ventana es menor?

▶ Respuesta

6. ¿Por qué duplicar el ancho de banda no cambia la propagación? ¿Y por qué duplicar la distancia no cambia la transmisión?

▶ Respuesta

7. ¿Qué es el jitter, y por qué un retardo alto y constante es mejor para el audio que uno medio y variable?

▶ Respuesta

8. En el Ejercicio 12, si duplico la distancia y duplico el ancho de banda a la vez, ¿se cancelan?

▶ Respuesta

🔗 Referencias

Exámenes - `eirc1103` / `sirc1103` (21-feb-2011) — **P5 (10 pts): retardo end-to-end con 3 enlaces y 2 conmutadores, y el caso cut-through**. Resuelto en el §3.1 de esta clase. - `sirc0912` (18-dic-2009) — P1 (10 pts): componentes del retardo, ancho de banda vs. velocidad de propagación, congestión. **Clase 5 §3.1.** - `sirc1103` — P2 (6 pts): congestión, pérdidas y retardo. **Clase 5 §3.2.** - `sirc1002` (26-feb-2010) — P5 (8 pts): jitter vs. retardo y sensibilidad de las aplicaciones. - `sirc0903` (20-feb-2009) — P5 (6 pts): full duplex vs. half duplex. **Clase 7.** - `sirc0707` (31-jul-2007) — P1 (10 pts): capacidad máxima sobre línea telefónica. **Clase 7.**

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., cap. 1 §1.5 "Delay, Loss, and Jitter in Packet-Switched Networks" (pág. ~10 a ~35 del PDF de la carpeta).

RFCs - `RFC 7323` — *Window Scale* (2014): lo que permite cubrir el BDP. - `RFC 5681` — *TCP Congestion Control* (2009): los cuatro algoritmos que reaccionan a la pérdida. - `RFC 3168` y `RFC 9768` — ECN y AccECN: marcar en vez de descartar, para que el objetivo sea evitar la cola. - `RFC 9331` — *L4S* (2023): ECN de umbral bajo. `RFC 9000` — *QUIC* (2021). `draft-ietf-ccwg-bbr` — BBRv3 (*Experimental*, **nunca fue RFC**).

Clases relacionadas - Clase 5 (teoría del retardo) · Clase 7 (Nyquist, Shannon y la capa física **+ Simulacro 1**) · Clase 19 (BDP y ventana de TCP) · Clase 21 (control de congestión y ECN).